

Gilgamesh — Project Progress

Step-by-step development record: from an empty Next.js template to a complete AI chat workspace.

Phase Summary

#	Step / Commit	Date	Type	Result
S1	Initial commit from Create Next App	2026-09-12	Scaffolding	Next.js app skeleton (App Router)
S2	Steps 1–4: Chat UI + management	2026-09-12	Feature	Working chat UI, themes, localStorage, settings
S3	Prisma + Supabase Postgres datasource	2026-09-12	Infrastructure	ORM wired to hosted Postgres
S4	Initial schema: User / Conversation / Message	2026-09-12	Data	Migrations applied to Supabase
S5	Supabase email/password auth	2026-09-12	Feature	Auth gate + sync-user provisioning
S6	Key rename + About section cleanup	2026-09-12	Polish	Safe publishable key handling, attribution
S7	Postgres persistence + Gemini AI responses	2026-09-12	Core	Real AI chat, multi-device history, regenerate
S8	Markdown rendering + typing dots	2026-09-13	UX	Rich assistant output + thinking animation
S9	Pitch-black UI + instant interactions	2026-09-13	Performance	Optimistic New Chat, no page spinner, #000 theme

S1 — Project Scaffolding COMPLETED

commit bcd36ba · 2026-09-12 17:27 · "Initial commit from Create Next App"

Created the project with Next.js's standard starter, giving a TypeScript + Tailwind CSS + App Router base. Confirmed `npm run dev` boots on `localhost:3000`. Planned the product name **Gilgamesh** and the feature set (chat, conversations, themes, settings, AI replies, persistence).

S2 — Chat Interface, Conversation Management, Themes, localStorage, Settings COMPLETED

commit ed508f0 · 2026-09-12 19:38 · "Steps 1-4: chat UI, conversation management, pure-black + light themes, localStorage persistence, settings"

Built the complete client-side chat experience without any backend yet:

- **Chat UI** — sidebar, header, message bubbles, composer with Send, and a three-dot thinking indicator.
- **Conversation management** — New chat, switching between chats, delete single, clear all, regenerate last answer, custom titles.
- **Themes** — a pure-black appearance plus a light appearance; brand mark component.
- **localStorage persistence** — conversations survived reloads before any database existed.
- **Settings** — appearance picker, enter-to-send toggle, About section.

Verified in the browser at 1280×800 and mobile widths (390/360 px): admin-less flows, no overflow, reload keeps data.

S3 — Prisma ORM with Supabase Postgres COMPLETED

commit 16f3500 · 2026-09-12 20:27 · "Add Prisma 6 with Supabase Postgres datasource configuration"

Added `prisma@6.19.3` pinned to Prisma 6 (version-locked to avoid breaking client/schema changes), created the `datasource db` block pointing at the hosted Supabase Postgres via connection-pooling (`DATABASE_URL`) and direct connection (`DIRECT_URL`), and moved credentials into `.env`. Verified `prisma generate` and connectivity from `prisma migrate dev status`.

S4 — Initial Database Schema COMPLETED

commit 034a669 · 2026-09-12 21:16 · "Add initial Gilgamesh chat schema: User, Conversation, Message models with indexes and cascade FKs"

Defined three models for the chat domain and applied the migration to Supabase Postgres:

```
model User { id, supabaseId @unique, email @unique, name?, createdAt, updatedAt, conversations[] }
model Conversation { id, userId FK->User, title, createdAt, updatedAt, user, messages[] } //
@@index([userId]), @@index([updatedAt])
model Message { id, conversationId FK->Conversation, role (user|assistant), content, createdAt }
// @@index([conversationId]), @@index([conversationId, createdAt]); cascade deletes on both relations
```

Ownership is enforced per-user (every row carries the owning `userId`), and ordered history reads are covered by the composite `(conversationId, createdAt)` index.

S5 — Supabase Email/Password Authentication COMPLETED

commit 2ff212d · 2026-09-12 21:38 · "Add Supabase email/password auth with auth gate, sync-user route, and supabaseId on User"

Guarded the app with real accounts: a browser-side Supabase client manages sign-in/sign-up and sessions; signed-out users see only the **AuthScreen**. A server-side `AuthGate` validates the JWT on every API call (expired sessions return 401 and are handled in the UI). A `GET /api/sync-user` route provisions the matching Prisma `User` row from the Supabase identity so every conversation belongs to a real account. Tested the full loop with a real Supabase-hosted account.

S6 — Key Handling Cleanup + About Section COMPLETED

commit d18c6ff · 2026-09-12 21:51 · "Rename Supabase key env var to NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY, remove demo mode badge, add About section with attribution and Gemini API note"

Renamed the Supabase key variable to `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` for correct publishable-key semantics, removed the temporary "demo mode" badge, and expanded the About dialog with proper attribution and a note about the Gemini API. `GEMINI_API_KEY` remains server-side only (never exposed to the browser).

S7 — Postgres Persistence + Real Gemini AI Responses COMPLETED

commit e6a3465 · 2026-09-12 22:42 · "Move chat persistence to Supabase Postgres and add Gemini AI responses"

The biggest step: replaced localStorage with the database and connected the live LLM.

- **lib/gemini.ts** — `generateAssistantReply(history)` calling `gemini-3.6-flash` via the Google GenAI SDK, translating app roles to model roles, with a 30 s timeout and typed `GeminiError`s for missing key, timeout, rate-limit (429), and empty responses.
- **API route set** — conversations list/create/update/delete, message append/delete, and the multi-turn `POST /[id]/chat` that persists the user turn, calls Gemini with full ordered history, saves the assistant reply, and dedupes network/DB writes with a request key.
- **ChatInterface** — the client now treats messages as DB-backed (completed via mappers), regeneration drops the last assistant reply and re-asks, and the sidebar reflects the latest conversation/date.

Verified end-to-end in the browser: sign-in, ask the model, reload — history restored from Postgres; regenerated answers replaced cleanly; expired session produced a correct 401 flow.

S8 — Markdown Rendering + Three-Dot Thinking Animation COMPLETED

commit 1d942ac · 2026-09-13 09:41 · "Render assistant responses as Markdown and add simple three-dot thinking animation"

Replaced plain-text assistant output with proper formatting using `react-markdown` + `remark-gfm`: headings, bullet/numbered lists, bold and italic, inline code, fenced code blocks with a copy button, and tables. The thinking indicator became simple staggered `• • •` CSS opacity dots (160 ms entrance transitions, disabled under `prefers-reduced-motion`). Verified via TypeScript, ESLint, a production build, and browser checks at multiple widths.

S9 — Pitch-Black UI + Instant, Optimistic Interactions LAST STEP

commit 7d82a97 · 2026-09-13 10:02 · "Give the chat UI a pitch-black theme and make New Chat and conversation switching feel instant"

Final polish focused on speed and a clean, minimal look:

- **Pitch-black palette** — background `#000`, sidebar `#050505`, composer & user bubbles `#111`, borders `#222`, text white/grey; bronze `#c9a876` reserved for small accents only. No gradients, glow, glassmorphism, or large radius — verified by computed styles.
- **Instant New Chat** — the new conversation is created optimistically in state (local id), rendered immediately, and the database create runs in the background; the local id is swapped for the server id when it resolves, so a user can even type and send before the create finishes.
- **Fast conversation switching** — the full-page "messages loading" spinner was removed. Known messages render instantly; missing history hydrates via a background `GET` that never clobbers a conversation currently generating. Conversation IDs are deduplicated so nothing loads twice.
- **Delete/clear safety** — pending optimistic conversations are resolved before deletion so no orphaned rows remain.

Verification: `tsc`, `eslint`, `next build` all pass; browser checks confirm palette values (rgb 0/0/0, 5/5/5, 17/17/17, 34/34/34), instant empty-chat creation with composer focus, background `POST /api/conversations`, immediate user bubble + dots on send, and no overflow at 1280/768/390/360 px.

Quality Gates Run Throughout

Gate	Command	Outcome
Static type check	<code>npx tsc --noEmit</code>	Passed after every feature step
Lint	<code>npm run lint</code>	Passed, zero rule violations
Production build	<code>npm run build</code>	Passed on every release step
Browser verification	local preview 1280×800 + 768/390/360 px	No overflow, reduced-motion respected
Live AI round-trip	real Gemini call through the API route	Response persisted and restored after reload

Environment & Toolchain

Item	Value
OS / runtime	Windows 11 · Node.js v24.19.0 · npm
Framework	Next.js 16.3.5 (App Router, Turbopack dev/build)
Languages	TypeScript (strict), React 19.2.8
Styling	Tailwind CSS v4 (CSS-first config, @theme tokens)
Database	PostgreSQL hosted on Supabase, connected via Prisma ORM 6.19.3
Auth	Supabase Auth (email/password) with JWT-validating server gate
AI	Google Gemini API — gemini-3.6-flash via @google/genai 2.22.0
Rendering	react-markdown 10 + remark-gfm 4 · lucide-react icons
Repository	github.com/mrkasif/Gilgamesh — branch master, 9 commits, working tree clean